

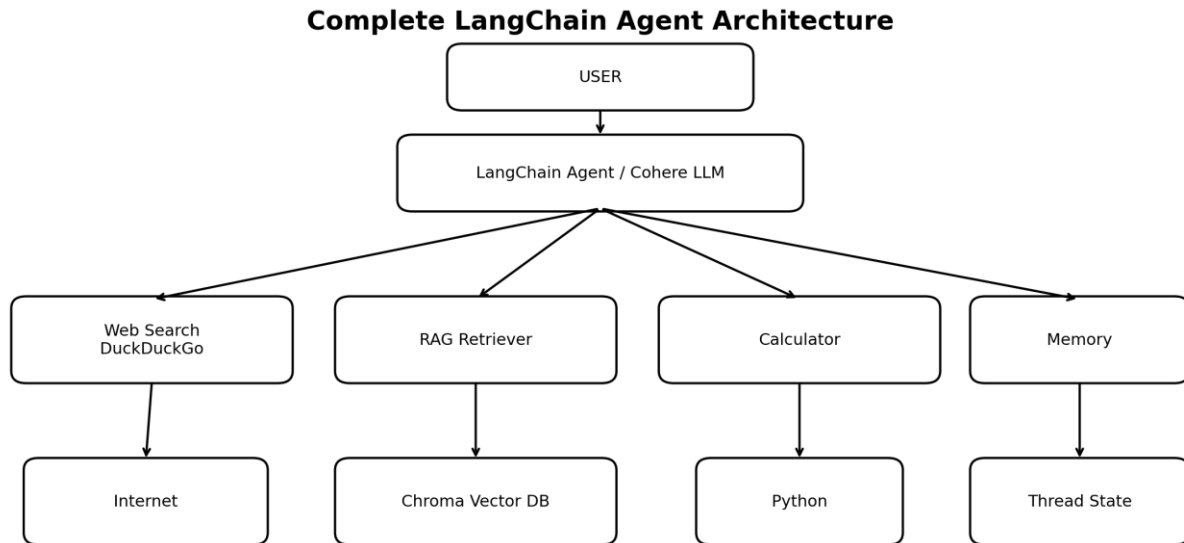
Complete LangChain AI Agent

Cohere + Tools + RAG + Memory

Hands-on Lab Manual

1. Objective and Architecture

Build a complete LangChain AI Agent using Cohere as the LLM, external tools for action, RAG for private document knowledge, and short-term conversation memory. This lab demonstrates the progression from a tool-calling agent toward an agentic AI workflow.



2. Components

Component	Purpose
Cohere Chat Model	Main reasoning/model layer
DuckDuckGo	Current/general web search
Calculator	Custom Python action
RAG Tool	Questions over private documents
Chroma	Vector database
Cohere Embeddings	Semantic embeddings for retrieval
Memory	Thread-scoped conversation state

3. Install the Required Packages

```
mkdir langchain-agentic-ai
cd langchain-agentic-ai
```

```
python -m venv venv
venv\Scripts\activate
```

```
pip install -U langchain
pip install -U langchain-community
pip install -U langchain-cohere
pip install -U langchain-chroma
pip install -U langgraph
pip install -U pypdf
pip install -U duckduckgo-search
```

Set the Cohere API key:

```
set COHERE_API_KEY=your_cohere_api_key
```

PowerShell:

```
$env:COHERE_API_KEY="your_cohere_api_key"
```

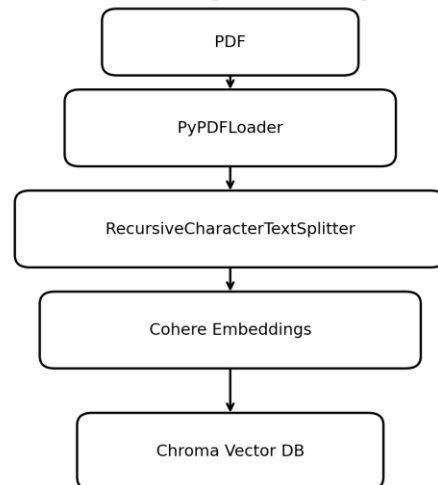
4. Suggested Project Structure

```
langchain-agentic-ai/  
├── documents/  
│   └── company_policy.pdf  
├── chroma_db/  
├── ingest.py  
├── test_rag.py  
└── agent.py
```

5. Build the RAG Knowledge Base

The RAG pipeline loads the PDF, splits it into chunks, converts chunks to embeddings, and stores them in Chroma.

RAG Knowledge Base Pipeline



```
from langchain_community.document_loaders import PyPDFLoader  
from langchain_text_splitters import RecursiveCharacterTextSplitter  
from langchain_cohere import CohereEmbeddings  
from langchain_chroma import Chroma  
  
loader = PyPDFLoader("documents/company_policy.pdf")  
documents = loader.load()  
  
print(f"Pages loaded: {len(documents)}")  
  
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=800,  
    chunk_overlap=150  
)  
  
chunks = text_splitter.split_documents(documents)  
  
print(f"Chunks created: {len(chunks)}")  
  
embeddings = CohereEmbeddings(  
    model="embed-v4.0"  
)
```

```

vector_store = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    collection_name="company_documents",
    persist_directory="./chroma_db"
)

print("Knowledge base created successfully.")

```

6. Test Retrieval Before Building the Agent

```

from langchain_cohere import CohereEmbeddings
from langchain_chroma import Chroma

embeddings = CohereEmbeddings(
    model="embed-v4.0"
)

vector_store = Chroma(
    collection_name="company_documents",
    embedding_function=embeddings,
    persist_directory="./chroma_db"
)

retriever = vector_store.as_retriever(
    search_kwargs={"k": 3}
)

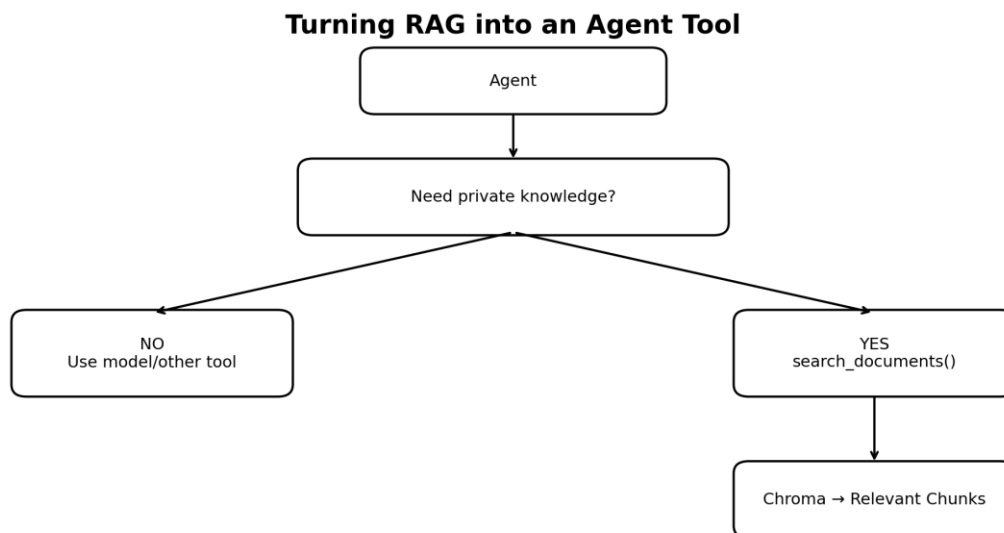
documents = retriever.invoke(
    "What is the leave policy?"
)

for document in documents:
    print(document.page_content)
    print("-" * 60)

```

7. Turn RAG into an Agent Tool

Instead of executing retrieval for every question, expose retrieval as a tool. The agent can then decide when private document knowledge is needed.



```

from langchain_core.tools import tool

@tool
def search_documents(query: str) -> str:
    """
    Search the organization's private documents.

    Use this tool when the user asks about information
    contained in company policies, internal documents,
    manuals, or uploaded knowledge-base documents.
    """

    documents = retriever.invoke(query)

    if not documents:
        return "No relevant information found."

    result = ""

    for document in documents:
        result += document.page_content + "\n\n"

    return result

```

8. Add Web Search and Calculator

```

from langchain_community.tools import DuckDuckGoSearchRun

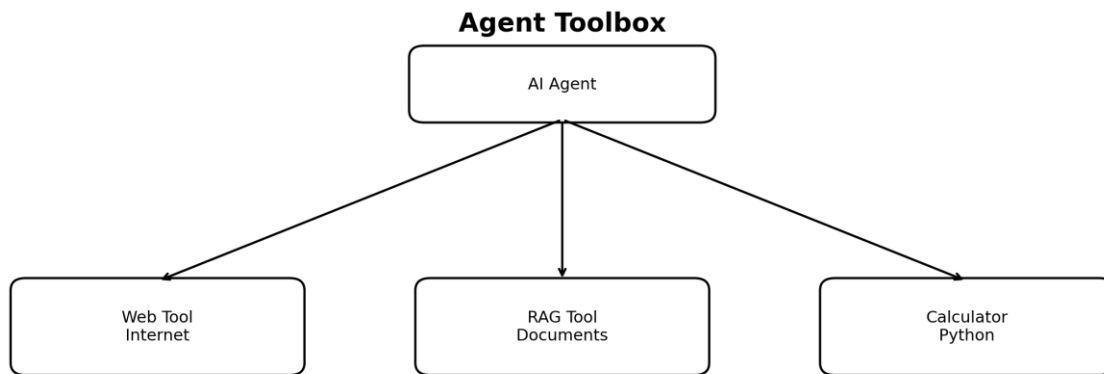
web_search = DuckDuckGoSearchRun()

@tool
def calculator(
    a: float,
    b: float,
    operation: str
) -> float:
    """Perform arithmetic calculations."""

    if operation == "add":
        return a + b
    elif operation == "subtract":
        return a - b
    elif operation == "multiply":
        return a * b
    elif operation == "divide":
        if b == 0:
            return "Cannot divide by zero"
        return a / b

    return "Unknown operation"

```



9. Create the Cohere Model and Agent

```
from langchain.agents import create_agent
from langchain_cohere import ChatCohere

tools = [
    web_search,
    search_documents,
    calculator
]

model = ChatCohere(
    model="command-a-03-2025",
    temperature=0
)

agent = create_agent(
    model=model,
    tools=tools,
    system_prompt="""
    You are an intelligent enterprise AI assistant.

    Use search_documents for internal organizational knowledge.
    Use web_search for current external information.
    Use calculator for arithmetic.

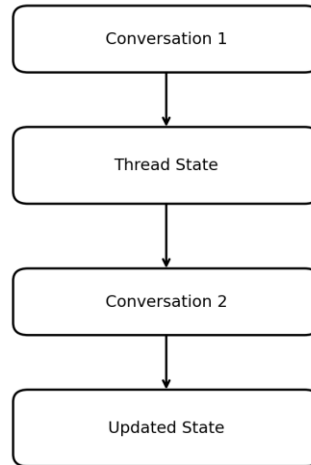
    Select tools automatically.
    You may call several tools when necessary.

    If private documents do not contain the requested
    information, clearly state that instead of inventing it.
    """
)
```

10. Add Short-Term Memory

LangChain agents can use a checkpointer so the same thread maintains state across multiple interactions.

Short-Term Memory / Thread State



```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()

agent = create_agent(
    model=model,
    tools=tools,
    system_prompt="""
    You are an intelligent enterprise AI assistant.

    Use private document search for organizational knowledge.
    Use web search for current information.
    Use calculator for calculations.
    Remember relevant context from the current conversation.
    """,
    checkpointer=checkpointer
)

config = {
    "configurable": {
        "thread_id": "user-001"
    }
}
```

11. Memory Test

```
response = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "My name is Ravi."
            }
        ]
    },
    config=config
)

response = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "What is my name?"
            }
        ]
    }
```

```

    ]
    },
    config=config
)

```

12. Complete Program

```

from langchain.agents import create_agent
from langchain_cohere import ChatCohere, CohereEmbeddings
from langchain_chroma import Chroma
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.tools import tool
from langgraph.checkpoint.memory import InMemorySaver

```

```

# =====
# EMBEDDINGS
# =====

```

```

embeddings = CohereEmbeddings(
    model="embed-v4.0"
)

```

```

# =====
# VECTOR DATABASE
# =====

```

```

vector_store = Chroma(
    collection_name="company_documents",
    embedding_function=embeddings,
    persist_directory="./chroma_db"
)

```

```

retriever = vector_store.as_retriever(
    search_kwargs={"k": 4}
)

```

```

# =====
# RAG TOOL
# =====

```

```

@tool
def search_documents(query: str) -> str:
    """
    Search internal documents and policies.

    Use this when questions relate to
    private organizational knowledge.
    """

    documents = retriever.invoke(query)

    if not documents:
        return "No relevant information found."

    result = ""

    for document in documents:
        result += document.page_content + "\n\n"

    return result

```

```

# =====
# WEB SEARCH TOOL
# =====

```

```

web_search = DuckDuckGoSearchRun()

```

```

# =====
# CALCULATOR
# =====

```

```

@tool
def calculator(
    a: float,
    b: float,
    operation: str
) -> float:
    """Perform arithmetic calculations."""

    if operation == "add":
        return a + b

```



```

        elif operation == "subtract":
            return a - b
        elif operation == "multiply":
            return a * b
        elif operation == "divide":
            if b == 0:
                return "Cannot divide by zero"
            return a / b

    return "Unknown operation"

# =====
# TOOLS
# =====

tools = [
    web_search,
    search_documents,
    calculator
]

# =====
# COHERE MODEL
# =====

model = ChatCohere(
    model="command-a-03-2025",
    temperature=0
)

# =====
# MEMORY
# =====

checkpointer = InMemorySaver()

# =====
# AGENT
# =====

agent = create_agent(
    model=model,
    tools=tools,
    checkpointer=checkpointer,
    system_prompt="""
    You are an intelligent enterprise AI assistant.

    Select tools automatically.

    Use search_documents when questions concern
    internal organizational information.

    Use web_search when current external
    information is required.

    Use calculator for arithmetic.

    You may call several tools when necessary.

    Remember conversation context.

    Do not fabricate information when a
    tool cannot find the answer.
    """
)

# =====
# CONVERSATION
# =====

config = {
    "configurable": {
        "thread_id": "user-001"
    }
}

while True:
    question = input("\nYou: ")

    if question.lower() in ["exit", "quit"]:
        break

    response = agent.invoke(
        {
            "messages": [

```

```

        {
            "role": "user",
            "content": question
        }
    ],
    config=config
)

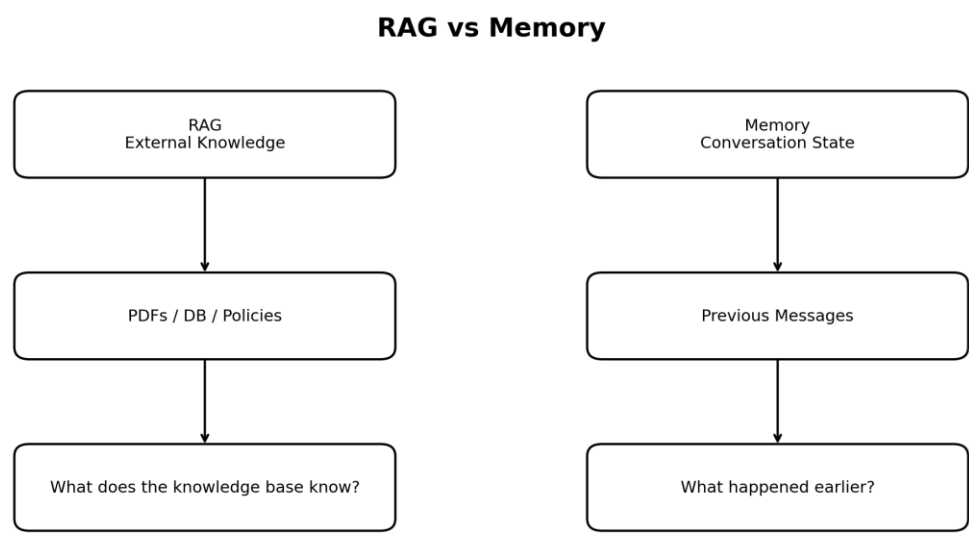
print(
    "\nAgent:",
    response["messages"][-1].content
)

```

13. Suggested Test Questions

- Explain machine learning in simple terms.
- What is 786 multiplied by 43?
- How many casual leave days are employees allowed?
- What are the latest developments in quantum computing?
- I am preparing a workshop on cybersecurity.
- Suggest an exercise relevant to the workshop I mentioned earlier.

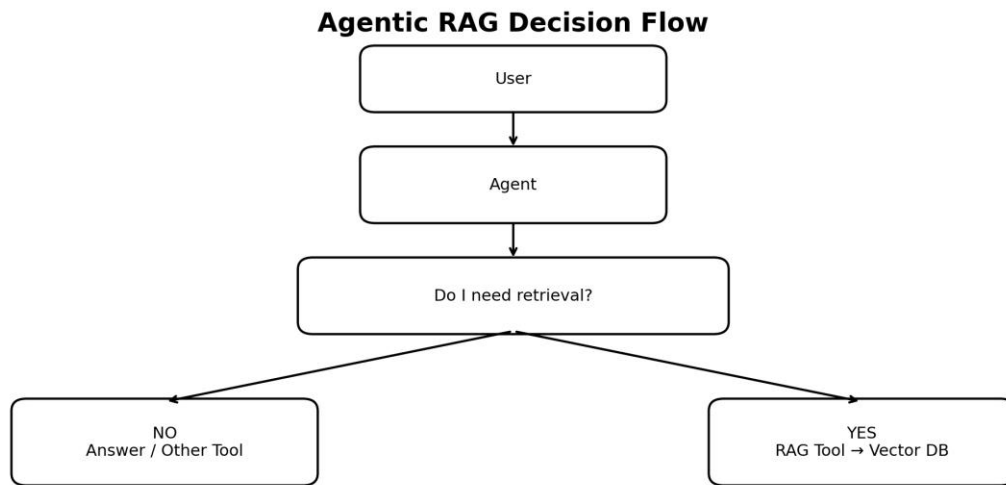
14. RAG vs Memory



RAG	Memory
External knowledge	Conversation/user state
PDFs, databases, policies	Previous messages
Usually a larger knowledge base	Usually thread/conversation-specific
Retrieved semantically	Stored/read from state
Example: company policy	Example: user's earlier request

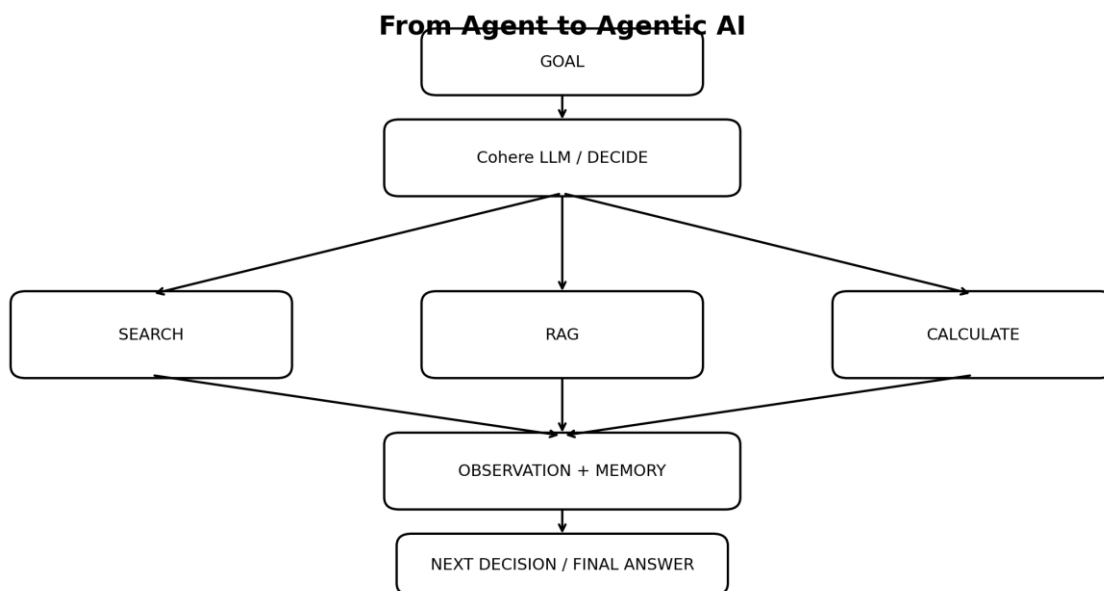
15. Traditional RAG vs Agentic RAG

Traditional RAG retrieves on every request. Agentic RAG allows the agent to decide whether retrieval is necessary.



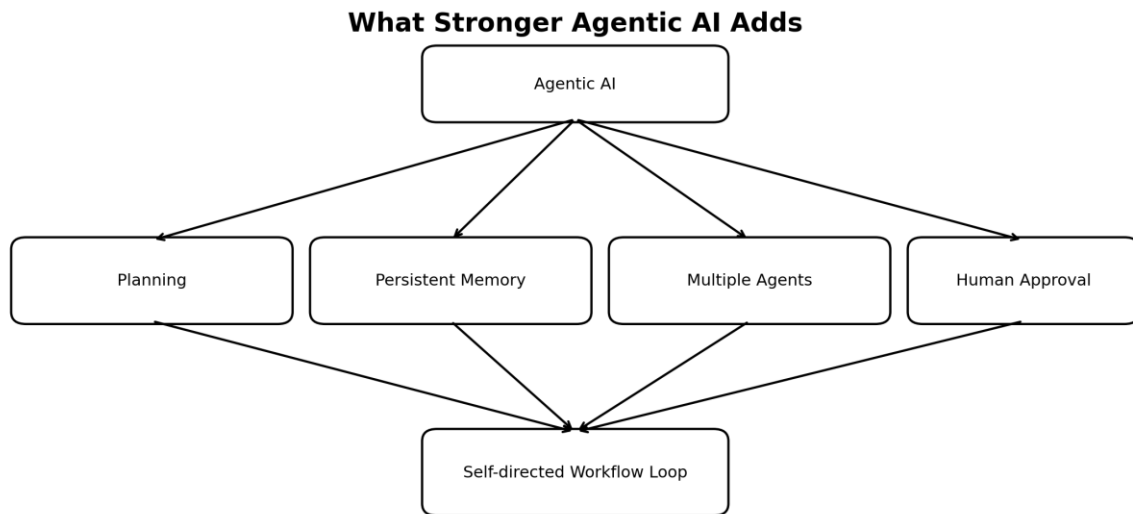
16. From Agent to Agentic AI

The application now has decision-making, tool selection, retrieval, external actions, state, and a repeated action-observation loop.



17. What Is Still Missing for Stronger Agentic AI?

A richer system can add planning, persistent long-term memory, multiple specialized agents, human approval, and explicit workflow control.



18. Learning Progression

Learning Progression



A useful progression is: LLM → tools → tool calling → agent → RAG → agentic RAG → memory/state → planning → multi-agent workflows → agentic AI.

19. Key Takeaways

- Tools give an LLM external capabilities.

- RAG gives the agent access to private or specialized knowledge.
- Memory preserves context across a conversation thread.
- Agentic RAG means retrieval is a decision, not a mandatory fixed step.
- The action-observation loop is a key foundation of agentic behavior.
- Planning, persistent memory, multiple agents, and approval steps move the system toward more advanced agentic AI.

20. Recommended Next Lab

Rebuild the same application using LangGraph with explicit nodes for Agent, RAG, Web Search, Calculator, Memory, Planner, Reviewer/Critic, Human Approval, and Final Answer. This makes the internal agentic workflow visible and controllable.